

|SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I 192511145 **sheik Mohamed m**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
Sheik Mohamed.m

Q1 (c) Compilation Phase for Each Error

Error Category Detected During

Lexical Error Lexical Analysis

Syntax Error Syntax Analysis (Parser)

Semantic Error Semantic Analysis

Logical Error During Program Execution/Testing

Runtime Error During Program Execution

Errors detected only during execution:

- Array index out of bounds
- Uninitialized pointer
- Division by zero
- Infinite loop

The screenshot shows the Programiz Online Compiler interface. On the left, the C code in `main.c` is displayed with line numbers 41 to 57. The code includes a pointer declaration, a loop to print numbers 1 to 5, and a calculation of the average of numbers 1 to 5. On the right, the 'Output' panel shows the execution results: 'Length is greater than 5', '80', 'Option 2', '10', 'Average = 20.00', and the numbers 1 through 5. At the bottom of the output, it says '=== Code Execution Successful ==='. A 'Programiz PRO >' button is visible in the top right corner.

Q2 – Answer

(a) Identify all syntax errors in the program

Line No.	Syntax Error	Reason
88	<code>int maximum(int x, int y</code>	Missing closing <code>)</code> before <code>{</code> .
103	<code>printf("Maximum is 80\n")</code>	Missing <code>;</code> after <code>printf</code> .

Line No.	Syntax Error	Reason
105	case 90	Missing : after case 90.
115	while(max > 75);	Incorrect placement of while in the do-while loop. It should come after the closing } of the do block.
117	printf("Limit Reached\n"	Missing closing) and ;.
119	main() not closed	Missing closing } for the main() function.

(b) Classification of Syntax Errors

Error	Category
Missing) in function definition	Parenthesis Error
Missing ; after printf	Missing Delimiter
Missing : after case 90	Switch-case Syntax Error
Incorrect do-while structure	Loop Syntax Error
Missing) and ; in printf	Function Call Syntax Error
Missing closing }	Block Delimiter Error

The screenshot shows the Programiz C Online Compiler interface. The code editor contains a C program with several syntax errors highlighted in red. The output window shows the program's execution results, including the maximum value and the limit reached.

```

main.c
26  default:
27      printf("Other Value\n");
28  }
29
30  do
31  {
32      printf("%d\n", max);
33      max--;
34  } while(max > 75);
35
36  if(max == 75)
37  {
38      printf("Limit Reached\n");
39  }
40
41  return 0;
42  }
  
```

Output:

```

Maximum is 80
80
79
78
77
76
Limit Reached
=== Code Execution Successful ===
  
```

(d) How does the parser detect syntax errors?

The **parser (syntax analyzer)** checks whether the program follows the grammatical rules of the C language.

- If it finds missing symbols such as ;,), }, or :, it reports a **syntax error**.
- **Error recovery** allows the parser to skip the incorrect part or insert missing symbols so that it can continue checking the remaining program instead of stopping immediately.

(e) Two improvements for better syntax error messages

1. **Provide meaningful error messages** that clearly indicate the line number and expected symbol (such as ;,), or }).
2. **Suggest possible corrections** (for example, "Did you forget a semicolon?" or "Expected ':' after case label").

Question 3:

(a) Identify all the errors in the program

Line No.	Error	Error Type	Reason
147	int @num = 5;	Lexical	@ is an invalid character in an identifier.
148	Missing ; after int value = 10	Syntax	Every statement must end with a semicolon.
150	factorial() called without argument	Semantic	Function requires one integer argument.
161	case 120	Syntax	Missing : after case 120.
168	numbers[5]	Runtime	Array index out of bounds (valid index: 0–4).
170	*ptr	Runtime	Pointer is uninitialized.
173	a / b where b = 0	Runtime	Division by zero.

Line No.	Error	Error Type	Reason
174	result + 0 + value	Code Optimization	Adding 0 is an unnecessary operation.
178	while loop has no i++	Logical/Runtime	Causes an infinite loop.

(b) Classification According to Compiler Phase

Compiler Phase	Error
Lexical Analysis	Invalid identifier @num
Syntax Analysis	Missing ;, Missing : after case 120
Semantic Analysis	factorial() called without required argument
Intermediate Code Generation	Expression result + 0 + value contains unnecessary operation
Code Optimization	Compiler can simplify result + 0 + value to result + value
Runtime	Array out of bounds, Uninitialized pointer, Division by zero, Infinite loop

(c) Explain how the compiler processes the program

- **Lexical Analysis:** Detects invalid tokens such as @num.
- **Syntax Analysis:** Detects missing symbols like ; and :.
- **Semantic Analysis:** Verifies function calls and argument matching; reports factorial() without an argument.
- **Intermediate Code Generation:** Produces intermediate code after successful analysis and can identify redundant expressions.
- **Code Optimization:** Removes unnecessary computations such as adding 0.
- **Runtime:** Errors such as array index out of bounds, uninitialized pointer access, division by zero, and infinite loops are detected only during program execution.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation